Requirements Table

Smart Expense & Budget Management System

1. Functional Requirements

ID	Requirement	Priority	Acceptance Criteria	Rationale	Comments
FR01	The system shall allow users to create an account and log in.	High	Pass: a new account is created and the user is taken to the dashboard after login. Fail: login works with a wrong password.	Nothing else in the app works until a user can get in.	Ties to SR01 and SR02.
FR02	The system shall allow users to add, edit and delete income records.	High	Pass: an income entry is saved, edited and deleted, and the available balance updates each time. Fail: deleted income still counts towards the balance.	Income is the base figure every budget calculation works from.	Say whether income can be recurring too.
FR03	The system shall allow users to add, edit and delete expense records.	High	Pass: an expense is saved with amount, date and category, and shows up in the list immediately. Fail: an expense saves with no amount or no date.	Core function of the app.	
FR04	The system shall allow users to categorize expenses.	High	Pass: expense is stored against the chosen category and shows under that category in the charts. Fail: expense is saved with no category at all.	All the analytics group by category, so this cannot be skipped.	Can users add their own categories?
FR05	The system shall automatically suggest a category for an expense.	Medium	Pass: on typing a merchant name the app shows a suggested category which the user can accept or change. Fail: the suggestion is applied without the user being able to change it.	Cuts down typing, which is the main reason people stop using expense apps.	Suggestion should never be forced on the user.
FR06	The system shall allow users to set monthly budgets.	High	Pass: a monthly limit is saved and shows on the dashboard for that month. Fail: a negative or zero budget is accepted.	This is the number everything else is compared against.	
FR07	The system shall allow users to set category-wise budgets.	Medium	Pass: separate limits can be set for categories like food or travel and each is tracked on its own. Fail: category budgets together exceed the monthly budget with no warning.	Overspending usually happens in one or two categories, not overall.	Should the app stop this or just warn?
FR08	The system shall calculate and display the remaining budget.	High	Pass: remaining budget = budget minus expenses, and it refreshes as soon as an expense is added. Fail: the figure is stale after adding or deleting an expense.	The single most looked-at number in the app.	
FR09	The system shall generate alerts when spending approaches or exceeds a budget.	High	Pass: an alert appears when spending crosses the warning level and again when the budget is crossed. Fail: budget is crossed with no alert shown.	Warning in time is the whole point, an alert after the month ends is useless.	Fix the threshold, e.g. 80%.

ID	Requirement	Priority	Acceptance Criteria	Rationale	Comments
FR10	The system shall display spending patterns using charts and graphs.	High	Pass: pie, bar and line charts load with the correct totals for the selected period. Fail: chart totals do not match the expense list.	Most users understand a chart faster than a table of numbers.	
FR11	The system shall detect unusual or abnormal spending patterns.	Medium	Pass: a transaction well outside the user's normal range for that category is flagged for review. Fail: an obvious outlier passes unflagged.	Catches duplicate charges and typing mistakes early.	How many past months are needed before this works?
FR12	The system shall predict the user's expected monthly expenditure.	Medium	Pass: a forecast for the current month is shown along with the past months it is based on. Fail: a forecast is shown for a user with almost no history.	Lets the user plan ahead instead of reacting at month end.	Needs a minimum history rule.
FR13	The system shall recommend budgets based on previous spending behaviour.	Medium	Pass: the app suggests a limit per category from past months and the user can accept or edit it. Fail: the suggested budget is higher than the user's income.	First-time budgeters do not know what a reasonable limit looks like.	
FR14	The system shall allow users to create and track savings goals.	Medium	Pass: a goal with a target amount is created and progress updates as money is saved. Fail: progress goes above 100% or turns negative.	Gives the user something to save towards, not just something to cut.	
FR15	The system shall estimate the time required to reach a savings goal.	Low	Pass: an estimated number of months is shown, based on the recent average saving rate. Fail: an estimate is shown when the user is saving nothing.	Makes the goal feel concrete and shows if the current rate is realistic.	Handle the zero-saving case.
FR16	The system shall allow users to record and track recurring expenses.	Medium	Pass: a recurring expense is created once and appears automatically in each following cycle. Fail: the same recurring expense is created twice in one cycle.	Rent and subscriptions are the expenses people forget to enter.	
FR17	The system shall provide reminders for upcoming recurring payments.	Medium	Pass: a reminder appears before the due date and stops once the payment is marked paid. Fail: reminders keep coming after payment is recorded.	Avoids late payment charges.	How many days before? Fix a number.
FR18	The system shall calculate a financial health score from spending and saving behaviour.	Low	Pass: a score is shown with a short reason for it, and it changes when spending habits change. Fail: the score stays the same across very different months.	A quick summary for users who do not want to read charts.	Explain how the score is worked out.
FR19	The system shall provide a what-if simulator for evaluating financial decisions.	Medium	Pass: entering a planned purchase shows the effect on the remaining budget and on savings goals. Fail: the simulated purchase is saved as a real expense.	Lets the user check a decision before making it, which is the main idea of the project.	Must clearly say it is a simulation.

ID	Requirement	Priority	Acceptance Criteria	Rationale	Comments
FR20	The system shall allow users to export their financial reports.	Low	Pass: a report downloads as PDF or CSV with the figures matching the dashboard. Fail: the exported file is empty or does not match the app.	Users want their own copy for records or for sharing.	

2. Non-Functional Requirements

ID	Requirement	Priority	Acceptance Criteria	Rationale	Comments
NFR01	Performance: the dashboard should load quickly under normal use.	High	Pass: dashboard with one year of data loads in under 3 seconds on a normal connection. Fail: it takes noticeably longer on a common device.	A slow dashboard is the fastest way to lose daily users.	The original wording said “a few seconds”. Fixed a number here.
NFR02	Usability: the interface should be simple enough for a first-time budgeter.	High	Pass: a new user can add an expense within one minute without help, in a small user test. Fail: testers cannot find how to add an expense.	The target users are people who found spreadsheets too complicated.	Testing with 5 users is enough at this stage.
NFR03	Availability: the app should be reachable whenever the user needs it.	Medium	Pass: uptime stays above 99% over a month of monitoring. Fail: repeated downtime during normal hours.	People check budgets at odd hours, usually right before spending.	
NFR04	Reliability: expense, budget, savings and balance calculations must be correct.	High	Pass: all figures match a manual calculation over a test set of 100 transactions. Fail: any mismatch in the totals.	One wrong balance and the user stops trusting the app completely.	Most important NFR for a finance app.
NFR05	Scalability: performance should hold as users and transactions grow.	Medium	Pass: response times stay within limits with 1000 users and 100000 transactions in a load test. Fail: response time roughly doubles under that load.	Transaction volume grows every month for every user.	
NFR06	Maintainability: the app should be built from modular components.	Medium	Pass: a feature such as the health score can be changed without editing unrelated modules. Fail: a small change forces edits across the codebase.	The prediction parts will change often, so they should be separable.	Hard to test, keep it as a design rule.
NFR07	Portability: the app should work on common desktop and mobile browsers.	Medium	Pass: all main screens work on current Chrome, Firefox and Safari, on desktop and mobile. Fail: a screen breaks on any of them.	Users switch between phone and laptop during the day.	
NFR08	Accuracy: predictions and calculations should be within an acceptable margin.	Medium	Pass: monthly forecast stays within 15% of actual spending for users with at least 3 months of history. Fail: forecasts are consistently further off.	A forecast nobody can rely on is worse than no forecast.	The 15% figure needs checking once we have data.

ID	Requirement	Priority	Acceptance Criteria	Rationale	Comments
NFR09	Responsiveness: the interface should adapt to different screen sizes.	Medium	Pass: layout works from a small phone width up to a desktop with no horizontal scrolling. Fail: content is cut off or overlaps at any common size.	Most entries will be made on a phone right after spending.	
NFR10	Recoverability: financial records should survive an app or server failure.	High	Pass: after a forced shutdown, all committed records are still present on restart. Fail: any record is lost.	Lost financial history cannot be recreated by the user.	Mention backup frequency.

3. Security Requirements

ID	Requirement	Priority	Acceptance Criteria	Rationale	Comments
SR01	The system shall require users to authenticate before accessing financial data.	High	Pass: requesting any data page without logging in redirects to login. Fail: any page shows data without a session.	All the data in this app is personal.	
SR02	Passwords shall be stored using secure hashing, not plain text.	High	Pass: the stored value for a password is a hash, not the password itself. Fail: passwords are readable in the database.	A leaked database should not leak passwords.	
SR03	The system shall use secure tokens and proper session management.	High	Pass: a token issued to one account is rejected when reused elsewhere after logout. Fail: an old token still works.	Stops a stolen session being reused.	
SR04	A user shall only be able to access their own income, expense, budget and savings data.	High	Pass: requesting another user’s record by changing the ID in the URL is refused. Fail: the record is returned.	This is the most likely mistake in a multi-user app.	Test this early.
SR05	The system shall validate user input before storing it.	High	Pass: script tags, oversized text and non-numeric amounts are all rejected. Fail: any of them are stored.	Protects against both bad data and injection attempts.	
SR06	Sensitive data shall be transmitted over HTTPS/TLS.	High	Pass: no request in the app goes over plain HTTP. Fail: any endpoint responds without TLS.	Financial data should never travel in the clear.	
SR07	The system shall log out or expire sessions after a period of inactivity.	Medium	Pass: an idle session is ended after the set time and the user must log in again. Fail: an idle session stays valid indefinitely.	Protects users on shared or public devices.	Decide the timeout, e.g. 15 minutes.
SR08	The system shall protect APIs from unauthorized access.	High	Pass: API calls without a valid token are refused and the attempt is logged. Fail: any endpoint responds without authentication.	The API is the real door into the data, not the UI.	

ID	Requirement	Priority	Acceptance Criteria	Rationale	Comments
SR09	The system shall rate limit repeated login and API attempts.	Medium	Pass: repeated failed logins from the same source are blocked after a set number of tries. Fail: unlimited attempts are allowed.	Basic protection against brute force attempts.	Fix the limit and the lockout time.
SR10	Financial data and credentials shall not be exposed in logs or error messages.	Medium	Pass: logs and error screens show no amounts, tokens or passwords. Fail: any of these appear in a stack trace.	Logs get shared during debugging and often end up in the wrong place.	

4. Traceability: User Story to Requirement to Use Case

User Story	Requirement	Use Case
US01	FR01, SR01	UC-01 Register & Login (includes Authenticate User)
US02	FR02	UC-02 Manage Income Records
US03, US04	FR03, FR04	UC-03 Manage Expense Records (includes Validate & Categorize Entry)
US05	FR05	Suggest Expense Category (extends UC-03)
US06, US07	FR06, FR07	UC-04 Manage Budgets
US08	FR08	UC-04 / UC-05
US09	FR09	Send Budget Alert (extends UC-04)
US10	FR10	UC-05 View Dashboard & Analytics
US11	FR11	Detect Unusual Spending (extends UC-03)
US12	FR12	Forecast Monthly Expenditure (included in UC-05)
US13	FR13	Recommend Budget (extends UC-04)
US14, US15	FR14, FR15	UC-06 Manage Savings Goals (includes Estimate Time to Goal)
US16	FR16	UC-07 Manage Recurring Expenses
US17	FR17	Send Payment Reminder (extends UC-07)
US18	FR18	UC-09 View Financial Health Score
US19	FR19	UC-08 Run What-If Simulation
US20	FR20	UC-10 Export Financial Report

Actors: User (primary). Analytics / Prediction Engine and Notification Service are supporting actors, since the category suggestion, anomaly detection, forecasting and alerting all run outside the main request flow.